Menus

Menus are one type of backend for Graphical User Interfaces, or GUIs; they handle the logic involved in interacting with some re to indirectly modify the internal data holder state. As such, a data holder should not be directly coupled to any menu, instead pas

`MenuType`

Menus are created and removed dynamically and as such are not registry objects. As such, another factory object is registered ins

`MenuType`s must be registered.

`MenuSupplier`

A `MenuType` is created by passing in a `MenuSupplier` and a `FeatureFlagSet` to its constructor. A `MenuSupplier` repre menu, and returns a newly created `AbstractContainerMenu`.

```
// For some DeferredRegister<MenuType<?>> REGISTER
public static final RegistryObject<MenuType<MyMenu>> MY_MENU = REGISTER.register("my_menu", () ->

// In MyMenu, an AbstractContainerMenu subclass
public MyMenu(int containerId, Inventory playerInv) {
  super(MY_MENU.get(), containerId);
  // ...
}
```

Note

The container identifier is unique for an individual player. This means that the same container id on two different players will r

The `MenuSupplier` is usually responsible for creating a menu on the client with dummy data references used to store and inter

`IContainerFactory`

If additional information is needed on the client (e.g. the position of the data holder in the world), then the subclass `IContain` provides a `FriendlyByteBuf` which can store additional information that was sent from the server. A `MenuType` can be creat

```
// For some DeferredRegister<MenuType<?>> REGISTER
public static final RegistryObject<MenuType<MyMenuExtra>> MY_MENU_EXTRA = REGISTER.register("my_m

// In MyMenuExtra, an AbstractContainerMenu subclass
public MyMenuExtra(int containerId, Inventory playerInv, FriendlyByteBuf extraData) {
  super(MY_MENU_EXTRA.get(), containerId);
  // Store extra data from buffer
  // ...
}
```

`AbstractContainerMenu`

All menus are extended from `AbstractContainerMenu`. A menu takes in two parameters, the `MenuType`, which represents t for the current accessor.

Important

The player can only have 100 unique menus open at once.

Each menu should contain two constructors: one used to initialize the menu on the server and one used to initialize the menu `MenuType`. Any fields that the server menu constructor contains should have some default for the client menu constructor.

```
// Client menu constructor
public MyMenu(int containerId, Inventory playerInventory) {
  this(containerId, playerInventory);
}

// Server menu constructor
public MyMenu(int containerId, Inventory playerInventory) {
  // ...
}
```

Each menu implementation must implement two methods: `#stillValid` and `#quickMoveStack`.

`#stillValid` and `ContainerLevelAccess`

`#stillValid` determines whether the menu should remain open for a given player. This is typically directed to the static `#s` attached to. The client menu must always return `true` for this method, which the static `#stillValid` does default to. This ir located.

A `ContainerLevelAccess` supplies the current level and location of the block within an enclosed scope. When constructing tl The client menu constructor can pass in `ContainerLevelAccess#NULL`, which will do nothing.

```
// Client menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory) {
  this(containerId, playerInventory, ContainerLevelAccess.NULL);
}

// Server menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory, ContainerLevelAccess access) {
  // ...
}

// Assume this menu is attached to RegistryObject<Block> MY_BLOCK
@Override
public boolean stillValid(Player player) {
  return AbstractContainerMenu.stillValid(this.access, player, MY_BLOCK.get());
}
```

Data Synchronization

Some data needs to be present on both the server and the client to display to the player. To do this, the menu implements a ba
synced to the client. For players, this is checked every tick.

Minecraft supports two forms of data synchronization by default: `ItemStack`s via `Slot`s and integers via `DataSlot`s. `Slot`
player in a screen, assuming the action is valid. These can be added to a menu within the constructor through `#addSlot` and `#`

> **Note**
>
> Since `Container`s used by `Slot`s are deprecated by Forge in favor of using the `IItemHandler` capability, the rest of the e

A `SlotItemHandler` contains four parameters: the `IItemHandler` representing the inventory the stacks are within, the inc
position of the slot will render on the screen relative to `AbstractContainerScreen#leftPos` and `#topPos`. The client men

In most cases, any slots the menu contains is first added, followed by the player's inventory, and finally concluded with the pl
upon the order of which slots were added.

A `DataSlot` is an abstract class which should implement a getter and setter to reference the data stored in the
`DataSlot#standalone`.

These, along with slots, should be recreated every time a new menu is initialized.

> **Warning**
>
> Although a `DataSlot` stores an integer, it is effectively limited to a **short** (-32768 to 32767) because of how it sends the value

```
// Assume we have an inventory from a data object of size 5
// Assume we have a DataSlot constructed on each initialization of the server menu

// Client menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory) {
  this(containerId, playerInventory, new ItemStackHandler(5), DataSlot.standalone());
}

// Server menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory, IItemHandler dataInventory, DataS
  // Check if the data inventory size is some fixed value
  // Then, add slots for data inventory
  this.addSlot(new SlotItemHandler(dataInventory, /*...*/));

  // Add slots for player inventory
  this.addSlot(new Slot(playerInventory, /*...*/));

  // Add data slots for handled integers
  this.addDataSlot(dataSingle);

  // ...
}
```

`ContainerData`

If multiple integers need to be synced to the client, a `ContainerData` can be used to reference the integers instead. T
`ContainerData`s can also be constructed in the data object itself if the `ContainerData` is added to the menu through `#`
interface. The client menu constructor should always supply a new instance via `SimpleContainerData`.

```
// Assume we have a ContainerData of size 3

// Client menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory) {
  this(containerId, playerInventory, new SimpleContainerData(3));
}

// Server menu constructor
public MyMenuAccess(int containerId, Inventory playerInventory, ContainerData dataMultiple) {
  // Check if the ContainerData size is some fixed value
  checkContainerDataCount(dataMultiple, 3);

  // Add data slots for handled integers
  this.addDataSlots(dataMultiple);
```

```
  // ...
}
```

Warning

As `ContainerData` delegates to `DataSlot`s, these are also limited to a **short** (-32768 to 32767).

`#quickMoveStack`

`#quickMoveStack` is the second method that must be implemented by any menu. This method is called whenever a stack has its previous slot or there is no other place for the stack to go. The method returns a copy of the stack in the slot being quick move

Stacks are typically moved between slots using `#moveItemStackTo`, which moves the stack into the first available slot. It take index (exclusive), and whether to check the slots from first to last (when `false`) or from last to first (when `true`).

Across Minecraft implementations, this method is fairly consistent in its logic:

```
// Assume we have a data inventory of size 5
// The inventory has 4 inputs (index 1 - 4) which outputs to a result slot (index 0)
// We also have the 27 player inventory slots and the 9 hotbar slots
// As such, the actual slots are indexed like so:
//   - Data Inventory: Result (0), Inputs (1 - 4)
//   - Player Inventory (5 - 31)
//   - Player Hotbar (32 - 40)
@Override
public ItemStack quickMoveStack(Player player, int quickMovedSlotIndex) {
  // The quick moved slot stack
  ItemStack quickMovedStack = ItemStack.EMPTY;
  // The quick moved slot
  Slot quickMovedSlot = this.slots.get(quickMovedSlotIndex)

  // If the slot is in the valid range and the slot is not empty
  if (quickMovedSlot != null && quickMovedSlot.hasItem()) {
    // Get the raw stack to move
    ItemStack rawStack = quickMovedSlot.getItem();
    // Set the slot stack to a copy of the raw stack
    quickMovedStack = rawStack.copy();

    /*
    The following quick move logic can be simplified to if in data inventory,
    try to move to player inventory/hotbar and vice versa for containers
    that cannot transform data (e.g. chests).
    */

    // If the quick move was performed on the data inventory result slot
    if (quickMovedSlotIndex == 0) {
      // Try to move the result slot into the player inventory/hotbar
      if (!this.moveItemStackTo(rawStack, 5, 41, true)) {
        // If cannot move, no longer quick move
        return ItemStack.EMPTY;
      }

      // Perform logic on result slot quick move
      slot.onQuickCraft(rawStack, quickMovedStack);
    }
    // Else if the quick move was performed on the player inventory or hotbar slot
    else if (quickMovedSlotIndex >= 5 && quickMovedSlotIndex < 41) {
      // Try to move the inventory/hotbar slot into the data inventory input slots
      if (!this.moveItemStackTo(rawStack, 1, 5, false)) {
        // If cannot move and in player inventory slot, try to move to hotbar
        if (quickMovedSlotIndex < 32) {
          if (!this.moveItemStackTo(rawStack, 32, 41, false)) {
            // If cannot move, no longer quick move
            return ItemStack.EMPTY;
          }
        }
        // Else try to move hotbar into player inventory slot
        else if (!this.moveItemStackTo(rawStack, 5, 32, false)) {
          // If cannot move, no longer quick move
          return ItemStack.EMPTY;
        }
      }
    }
    // Else if the quick move was performed on the data inventory input slots, try to move to pla
    else if (!this.moveItemStackTo(rawStack, 5, 41, false)) {
      // If cannot move, no longer quick move
      return ItemStack.EMPTY;
    }

    if (rawStack.isEmpty()) {
      // If the raw stack has completely moved out of the slot, set the slot to the empty stack
      quickMovedSlot.set(ItemStack.EMPTY);
    } else {
      // Otherwise, notify the slot that that the stack count has changed
      quickMovedSlot.setChanged();
```

```
    }

    /*
    The following if statement and Slot#onTake call can be removed if the
    menu does not represent a container that can transform stacks (e.g.
    chests).
    */
    if (rawStack.getCount() == quickMovedStack.getCount()) {
      // If the raw stack was not able to be moved to another slot, no longer quick move
      return ItemStack.EMPTY;
    }
    // Execute logic on what to do post move with the remaining stack
    quickMovedSlot.onTake(player, rawStack);
  }

  return quickMovedStack; // Return the slot stack
}
```

Opening a Menu

Once a menu type has been registered, the menu itself has been finished, and a screen has been attached, a menu can then be o server. The method takes in the player opening the menu, the `MenuProvider` of the server side menu, and optionally a `Frien`

> **Note**
>
> `NetworkHooks#openScreen` with the `FriendlyByteBuf` parameter should only be used if a menu type was created using

`MenuProvider`

A `MenuProvider` is an interface that contains two methods: `#createMenu`, which creates the server instance of the menu, a screen. The `#createMenu` method contains three parameter: the container id of the menu, the inventory of the player who ope

A `MenuProvider` can easily be created using `SimpleMenuProvider`, which takes in a method reference to create the server m

```
// In some implementation
NetworkHooks.openScreen(serverPlayer, new SimpleMenuProvider(
  (containerId, playerInventory, player) -> new MyMenu(containerId, playerInventory),
  Component.translatable("menu.title.examplemod.mymenu")
));
```

Common Implementations

Menus are typically opened on a player interaction of some kind (e.g. when a block or entity is right-clicked).

Block Implementation

Blocks typically implement a menu by overriding `BlockBehaviour#use`. If on the logical client, the interacti `InteractionResult#CONSUME`.

The `MenuProvider` should be implemented by overriding `BlockBehaviour#getMenuProvider`. Vanilla methods use this to

```
// In some Block subclass
@Override
public MenuProvider getMenuProvider(BlockState state, Level level, BlockPos pos) {
  return new SimpleMenuProvider(/* ... */);
}

@Override
public InteractionResult use(BlockState state, Level level, BlockPos pos, Player player, Interact
  if (!level.isClientSide && player instanceof ServerPlayer serverPlayer) {
    NetworkHooks.openScreen(serverPlayer, state.getMenuProvider(level, pos));
  }
  return InteractionResult.sidedSuccess(level.isClientSide);
}
```

> **Note**
>
> This is the simplest way to implement the logic, not the only way. If you want the block to only open the menu under `InteractionResult#PASS` or `#FAIL` if the conditions are not met.

Mob Implementation

Mobs typically implement a menu by overriding `Mob#mobInteract`. This is done similarly to the block implementation w spectator mode viewing.

```
public class MyMob extends Mob implements MenuProvider {
  // ...

  @Override
  public InteractionResult mobInteract(Player player, InteractionHand hand) {
    if (!this.level.isClientSide && player instanceof ServerPlayer serverPlayer) {
      NetworkHooks.openScreen(serverPlayer, this);
    }
    return InteractionResult.sidedSuccess(this.level.isClientSide);
  }
}
```

Note

Once again, this is the simplest way to implement the logic, not the only way.